

A Arquitetura da World Wide Web

Identificação de recursos (URIs), interação (protocolo HTTP) e representação

- Este é o **primeiro tema** do conteúdo programático da UC — base conceptual para todos os restantes
- *[PROFUNDIDADE ASSUMIDA]* — a FUC não detalha o nível de profundidade; assume-se introdução teórico-prática, alinhada com o facto de ser o primeiro contacto dos alunos com o desenvolvimento de aplicações distribuídas

Objetivos de aprendizagem

- Compreender os **três pilares arquiteturais** da Web: identificação, interação e representação
- Saber identificar e construir corretamente um **URI**
- Compreender o modelo **pedido-resposta** do protocolo **HTTP**
- Distinguir **recurso** de **representação de um recurso**

Os Três Pilares da Arquitetura da Web

A Web foi concebida segundo o relatório *Architecture of the World Wide Web, Volume One* (W3C TAG, 2004), que identifica três componentes fundamentais:

- **Identificação** — todo o recurso na Web é identificado de forma única através de um **URI** (Uniform Resource Identifier)
- **Interação** — agentes (clientes e servidores) trocam mensagens segundo o protocolo **HTTP**, tipicamente seguindo o estilo **REST** (Representational State Transfer)
- **Representação** — um recurso não é transmitido diretamente; é transmitida uma **representação** desse recurso, num formato concreto (HTML, JSON, imagem, etc.)

Conceito central: um *recurso* é uma noção abstrata (ex.: "a previsão do tempo para Lisboa hoje"); o que circula na rede são **representações** concretas desse recurso.

Identificação de Recursos: o conceito de URI

- **URI (Uniform Resource Identifier)**: cadeia de caracteres que identifica de forma **única e universal** um recurso
- Um URI pode ser:
 - **URL (Uniform Resource Locator)** — identifica o recurso e indica **como o localizar/aceder** (ex.: através de HTTP)
 - **URN (Uniform Resource Name)** — identifica o recurso por **nome**, sem indicar a sua localização
- Na prática, na Web, a quase totalidade dos URIs usados são **URLs**

Sintaxe genérica de um URI (RFC 3986):

```
esquema://autoridade/caminho?query#fragmento
```

Anatomia de um URI

Exemplo prático:

```
https://www.isel.pt:443/cursos/leic?ano=2025&semestre=2#programa
```

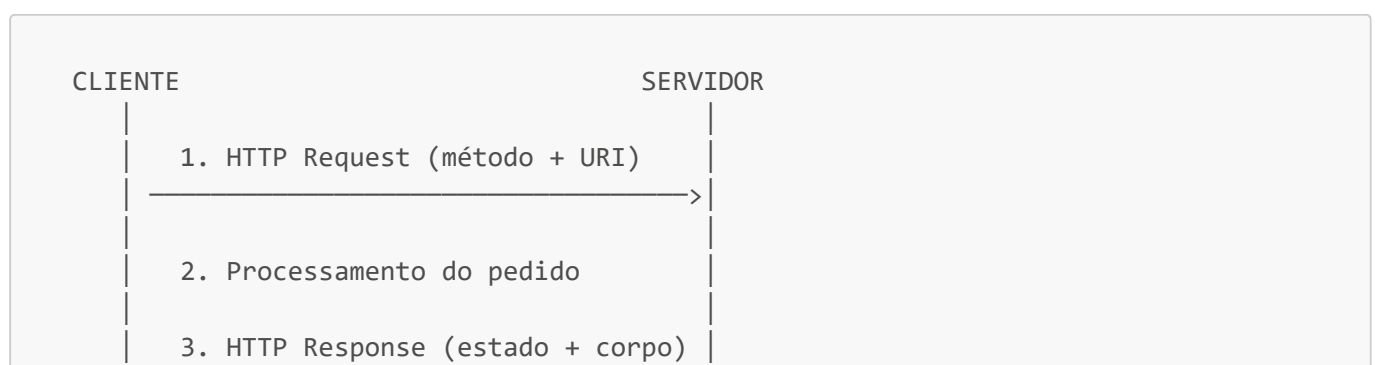
Componente	Valor	Função
esquema	https	Protocolo/mecanismo de acesso
autoridade	www.isel.pt:443	Host (+porta opcional)
caminho	/cursos/leic	Localização hierárquica do recurso
query	ano=2025&semestre=2	Parâmetros adicionais
fragmento	programa	Sub-parte da representação (interpretado no cliente)

- **Boas práticas:** URIs devem ser **opacos** para o cliente (não devem ser "interpretados"), **persistentes** no tempo e identificar **um único recurso**
- *[FONTE EXTERNA]* — a recomendação "Cool URIs don't change" é um princípio frequentemente associado ao design de URIs na Web, não citado explicitamente na bibliografia da FUC

Interação: o Protocolo HTTP

- **HTTP (HyperText Transfer Protocol):** protocolo de aplicação que define como **clientes** e **servidores** trocam mensagens sobre os recursos identificados por URIs
- Características fundamentais:
 - Modelo **cliente-servidor**: o cliente inicia sempre o **pedido (request)**, o servidor devolve a **resposta (response)**
 - Protocolo **stateless** (sem estado): cada pedido é processado de forma independente, sem memória de pedidos anteriores
 - Assenta tipicamente sobre **TCP/IP**
- Referência normativa na bibliografia da UC: **RFC 2616 — Hypertext Transfer Protocol HTTP/1.1**
- *[FONTE EXTERNA]* — atualmente a especificação HTTP/1.1 encontra-se atualizada pelas RFC 7230–7235 (e HTTP/2 e HTTP/3 como versões posteriores do protocolo), não referenciadas na bibliografia da FUC

O Ciclo Pedido-Resposta





- O cliente especifica **o quê** (URI do recurso) e **como** (método HTTP) quer interagir
- O servidor responde com um **código de estado** e, opcionalmente, uma **representação** do recurso no corpo da resposta

Métodos HTTP

Os métodos definem a **semântica da interação** pretendida sobre o recurso identificado pelo URI:

- **GET** — obter uma representação do recurso (não deve alterar o estado do servidor — **seguro**)
- **POST** — submeter dados para processamento/criação de um novo recurso
- **PUT** — criar ou substituir integralmente um recurso
- **DELETE** — remover um recurso
- **HEAD** — como GET, mas devolve apenas os cabeçalhos (sem corpo)
- **GET** e **HEAD** são **idempotentes e seguros**; **PUT** e **DELETE** são **idempotentes** mas não seguros; **POST** geralmente não é idempotente
- *[FONTE EXTERNA]* — a classificação detalhada de métodos como *seguros* e *idempotentes* surge de forma mais sistemática nas RFC 7231, mas o conceito está implícito na RFC 2616

Estrutura de um Pedido HTTP

Um pedido HTTP é composto por três partes:

```
GET /cursos/leic?ano=2025 HTTP/1.1
Host: www.isel.pt
Accept: text/html
User-Agent: Mozilla/5.0
```

- **Linha de pedido:** **método** **URI** **versão-HTTP**
- **Cabeçalhos (headers):** metadados sobre o pedido, o cliente e as preferências de representação
- **Corpo (body):** opcional — usado tipicamente em POST/PUT para enviar dados

Cabeçalhos relevantes para identificação e representação:

- **Host** — indica a autoridade do URI (essencial, já que um servidor pode alojar vários domínios)
- **Accept** — indica que **formatos de representação** o cliente está disposto a receber

Estrutura de uma Resposta HTTP

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 1542

<html>...</html>
```

- **Linha de estado:** **versão-HTTP** **código-de-estado** **frase-de-razão**
- **Cabeçalhos:** metadados sobre a resposta e a representação devolvida
- **Corpo:** a **representação** do recurso (pode estar ausente, ex.: em respostas 204)

Classes de códigos de estado:

Classe	Significado	Exemplos
1xx	Informativo	100 Continue
2xx	Sucesso	200 OK, 201 Created, 204 No Content
3xx	Redireção	301 Moved Permanently, 304 Not Modified
4xx	Erro do cliente	400 Bad Request, 404 Not Found
5xx	Erro do servidor	500 Internal Server Error

HTTP Stateless: o que significa na prática

- O servidor **não guarda informação** sobre pedidos anteriores do mesmo cliente
- Cada pedido HTTP deve conter **toda a informação necessária** para ser processado de forma autónoma
- **Vantagens:** simplicidade, escalabilidade (qualquer servidor pode responder a qualquer pedido), facilidade de *caching*
- **Implicação prática:** mecanismos de "estado de sessão" (ex.: login de utilizador) têm de ser implementados **por cima** do HTTP, recorrendo a mecanismos como **cookies**, não fazendo parte do protocolo base
- *[FONTE EXTERNA]* — mecanismos concretos de manutenção de sessão (cookies, tokens) pertencem a um tema posterior do programa da UC (componente de servidor) e são aqui apenas referidos como contexto

Representação: recurso vs. representação

- O **recurso** é o conceito abstrato identificado pelo URI; a **representação** é a sequência de bytes concreta transmitida na resposta
- Um mesmo recurso pode ter **múltiplas representações** — ex.: o mesmo URI pode devolver HTML, JSON ou XML, consoante o pedido
- Mecanismo que permite escolher a representação adequada: **negociação de conteúdo (content negotiation)**
 - Cabeçalho **Accept** (cliente) ↔ cabeçalho **Content-Type** (servidor)

Exemplo de negociação de conteúdo:

```
GET /cursos/leic HTTP/1.1
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json

{"curso": "LEIC", "ano": 2025}
```

- O **media type** (ex.: `text/html`, `application/json`, `image/png`) identifica de forma normalizada o formato da representação (registo IANA)

Exemplo Integrador

Pedido de um cliente ao identificar e interagir com um recurso da Web:

```
GET /alunos/12345 HTTP/1.1
Host: portal.isel.pt
Accept: application/json
```

Resposta do servidor com uma representação do recurso identificado:

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 58

{"id": 12345, "nome": "Jorge", "curso": "LEIC"}
```

- O **URI** `https://portal.isel.pt/alunos/12345` identifica univocamente o recurso "aluno 12345"
- O **método GET** define a interação pretendida (obter)
- O **corpo da resposta** é uma **representação** do recurso, no formato negociado (`application/json`)

Síntese

- A arquitetura da Web assenta em **três pilares articulados: identificação (URI), interação (HTTP) e representação**
- **URI**: identifica de forma única e universal um recurso; a sua sintaxe inclui esquema, autoridade, caminho, query e fragmento
- **HTTP**: protocolo cliente-servidor, **stateless**, baseado num ciclo **pedido-resposta**, com métodos que definem a semântica da interação (GET, POST, PUT, DELETE, ...)

- **Representação:** o que efetivamente circula na rede; um recurso pode ter várias representações, escolhidas por **negociação de conteúdo**
- Estes três conceitos constituem a **base teórica** sobre a qual assentam os temas seguintes da UC: distribuição de conteúdos web, criação de REST APIs e o desenvolvimento das componentes de cliente e servidor